ADA - Análisis y Diseño de Algoritmos, 2025-2
Tarea 4: Semanas 10 y 11

Para entregar el domingo 19 de octubre de 2025

Problemas conceptuales a las 23:59 por BrightSpace

Problemas prácticos a las 23:59 en la arena de programación

Tanto los ejercicios como los problemas deben ser resueltos, pero únicamente las soluciones de los problemas deben ser entregadas. La intención de los ejercicios es entrenarlo para que domine el material del curso; a pesar de que no debe entregar soluciones a los ejercicios, usted es responsable del material cubierto en ellos.

Instrucciones para la entrega

Para esta tarea y todas las tareas futuras, la entrega de soluciones es *individual*. Por favor escriba claramente su nombre, código de estudiante y sección en cada hoja impresa entregada o en cada archivo de código (a modo de comentario). Adicionalmente, agregue la información de fecha y nombres de compañeros con los que colaboró; igualmente cite cualquier fuente de información que utilizó.

¿Cómo describir un algoritmo?

En algunos ejercicios y problemas se pide "dar un algoritmo" para resolver un problema. Una solución debe tomar la forma de un pequeño ensayo (es decir, un par de párrafos). En particular, una solución debe resumir en un párrafo el problema y cuáles son los resultados de la solución. Además, se deben incluir párrafos con la siguiente información:

- una descripción del algoritmo en castellano y, si es útil, pseudo-código;
- por lo menos un diagrama o ejemplo que muestre cómo funciona el algoritmo;
- una demostración de la corrección del algoritmo; y
- un análisis de la complejidad temporal del algoritmo.

Recuerde que su objetivo es comunicar claramente un algoritmo. Las soluciones algorítmicas correctas y descritas *claramente* recibirán alta calificación; soluciones complejas, obtusas o mal presentadas recibirán baja calificación.

Problemas conceptuales

1. Ejercicio 2.3 (a): *Addition Chain* (Erickson, página 94).

Problemas prácticos

Hay cinco problemas prácticos cuyos enunciados aparecen a partir de la siguiente página.

A - Determine the Combination

Source file name: `combination.py`
Time limit: 1 second

Jahir is a student of NSU (*Nice Students' University*). He hates the chapter Permutation & Combination of the subject Discrete Math. But his teacher give him a assignment to generate all the r combination of a string. But he is too busy with his new girlfriend to do the assignment himself. So he went to Shabuj, a student of CSE (*Calculation Science and Engineering*) in BUET (*Bangladesh University of Extraordinary Talents*). He asked him to make a program to generate the combinations. But Shabuj is always lazy. He wants your help.

Your task is to print all different r combinations of a string s (a r combination of a string s is a collection of exactly r letters from different positions in s).

There may be different permutations of the same combination; consider only the one that has its r characters in non-decreasing order.

Input

The input is consist of several test cases. Each test case consists of a string s (the length of s is between 1 and 30) and an integer r ($0 < r \leq \ length\ of\ s$). The string consists of only lowercase letters. Any letter can occur more than once.

The input must be read from standard input.

Output

For each test case you have to print all different r combinations of s in lexicographic order in separate line.

The output must be written to standard output.

Sample Input	**Sample Output**

`abcde 2`
`abcd 3`
`aba 2`
	`ab`
	`ac`
	`ad`
	`ae`
	`bc`
	`bd`
	`be`
	`cd`
	`ce`
	`de`
	`abc`
	`abd`
	`acd`
	`bcd`
	`aa`
	`ab`

B - The Problem of the Crazy Linguist

Source file name: `linguist.py`
Time limit: 3 seconds

Please, help the crazy linguist! He is trapped in his madness. He has developed a "Spanish Beauty Criterion" for words. It is defined as follows. Given a word w

$$w = x_1 x_2 x_3 \dots x_n$$

(where n is the length of the word), he is interested in words which are formed by letters following pattern:

$$x_i \in \begin{cases} \{\text{bcdfghjklmnpqrstvwxyz}\} & \text{if } i \text{ is odd} \\ \{\text{aeiou}\} & \text{if } i \text{ is even.} \end{cases}$$

and, also, in his madness, he won't allow a word that actually has i, j, and k, so that $x_i = x_j = x_k$ (that is, any letter can appear in the word at most two times).

Then, the "Spanish Beauty Criterion" (SBC) is defined as:

$$SBC(w) = \sum_{i \in 1 \dots n} i \cdot P(x_i)$$

where P is defined as the probability of appearance of a letter in Spanish, defined by the following table:

a	b	c	d	e	f	g	h	i	j	k	l	m
12.53	1.42	4.68	5.86	13.68	0.69	1.01	0.70	6.25	0.44	0.00	4.97	3.15

n	o	p	q	r	s	t	u	v	w	x	y	z
6.71	8.68	2.51	0.88	6.87	7.98	4.63	3.93	0.90	0.02	0.22	0.90	0.52

So, given a word w, of size n, our poor linguist wants to know **if this word is above or below the average of the SBC of all the words of size n that can be constructed following the above pattern and start with the same letter than w.**

The input must be read from standard input.

Input

The input will have a first line with a number, N, the number of samples that will be entered to know if they are above or below the average. Following this first line, N lines with a word in each one (of at most seven characters each). All the input words follow the given pattern.

The input must be read from standard input.

Output

The output will have N lines, each line corresponding to one input word in order, showing just `'above or equal'` or `'below'`, depending on the value of the SBC of that word relative to the average of those of the same size.

The output must be written to standard output.

Sample Input	Sample Output
5 bubu terabit hacer qed loco	below above or equal above or equal above or equal above or equal

C - 23 Out of 5

Source file name: `out.py`
Time limit: 1 second

Your task is to write a program that can decide whether you can find an arithmetic expression consisting of five given numbers a_i ($1 \leq i \leq 5$) that will yield the value 23. For this problem we will only consider arithmetic expressions of the following from:

$$((((a_{\pi(1)} \oplus_1 a_{\pi(2)}) \oplus_2 a_{\pi(3)}) \oplus_3 a_{\pi(4)}) \oplus_4 a_{\pi(5)})$$

where $\pi : \{1, 2, 3, 4, 5\} \to \{1, 2, 3, 4, 5\}$ is a bijective function and $\oplus_i \in \{+, -, *\}$, with $1 \leq i \leq 4$.

Input

The input consists of 5-tuples of positive integers, each between 1 and 50. Input is terminated by a line containing five zero's. This line should not be processed.

The input must be read from standard input.

Output

For each 5-tuple output "`Possible`" (without the quotes) if there is an arithmetic expression (as described above) that yields 23. Otherwise, output "`Impossible`".

The output must be written to standard output.

Sample Input	**Sample Output**

`1 1 1 1 1`
`1 2 3 4 5`
`2 3 5 7 11`
`0 0 0 0 0`
	`Impossible`
	`Possible`
	`Possible`

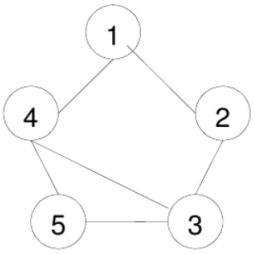
D - All Walks of Length n from the First Node

Source file name: `walks.py`
Time limit: 1 second

A computer network can be represented as a graph. Let $G = (V, E)$ be an undirected graph, $V = (v_1, v_2, v_3, \ldots, v_m)$ represents all nodes, where m is the number of nodes, and E represents all edges. The first node is v_1 and the last node is v_m. The number of edges is k. Define the adjacency matrix $A = (a_{ij})_{m \times m}$ where

$$a_{ij} = 1 \text{ if } \{v_i, v_j\} \in E, \text{ otherwise } a_{ij} = 0$$

An example of the adjacency matrix and its corresponding graph are as follows:



$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Calculate

$$A^n = A \cdot A \cdots A$$

and use the Boolean operations where $0 + 0 = 0$, $0 + 1 = 1 + 0 = 1$, $1 + 1 = 1$, and $0 \cdot 0 = 0 \cdot 1 = 1 \cdot 0 = 0$, $1 \cdot 1 = 1$. The entry in row i and column j of A^n is 1 if and only if at least there exists a walk of length n between the i-th and j-th nodes of V. In other words, the distinct walks of length n between the i-th and j-th nodes of V may be more than one. Note that the node in the paths can be repetitive.

The following example shows the walks of length 2

$$A^2 = A \cdot A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Write programs to do above calculation and print out all distinct walks of length n. (In this problem we let the maximum walks of length n be 5 and the maximum number of nodes be 10.)

Input

The input file contains a number of test examples, the test examples are separated by `'-9999'`. Each test example consists of the number of nodes and the length of walks in the first row, and then the adjacency matrix.

The input must be read from standard input.

Output

The output file must contain all distinct walks of the length n, and with all its nodes different, from the first node, listed in lexicographical order. In case there are not walks of length n, just print `'no walk of length n'`.

Separate the output of the different cases by a blank line.

The output must be written to standard output.

Sample Input	**Sample Output**
5 2 0 1 0 1 0 1 0 1 0 0 0 1 0 1 1 1 0 1 0 1 0 0 1 1 0 -9999 5 3 0 1 0 1 0 1 0 1 0 0 0 1 0 1 1 1 0 1 0 1 0 0 1 1 0	(1,2,3) (1,4,3) (1,4,5) (1,2,3,4) (1,2,3,5) (1,4,3,2) (1,4,3,5) (1,4,5,3)

E - Water Restrictions

Source file name: `water.py`
Time limit: 5 seconds

Due to the current water restrictions, the government has forbidden to waste more than C litres of water per house. The owner of a detached house must reduce his water consumption. He has a onedimensional garden of length L, with N sprinklers located at fixed positions. Each sprinkler, i, has a maximum possible flow, $m[i]$; it can be programmed to work at any current flow between 0 and $m[i]$. If the current flow is set to $c[i]$, then the sprinkler will irrigate between $c[i]$ positions to its left and $c[i]$ positions to its right.

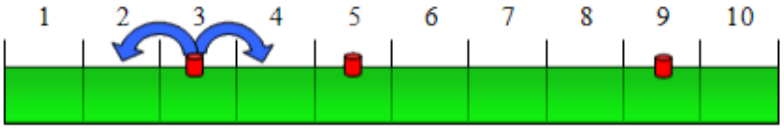


Figure 1: A sample one-dimensional garden of length $L = 10$, and three sprinklers at positions 3, 5 and 9. The first sprinkler is programmed to $c[1] = 1$.

In total, the sum of all $c[i]$ can never be greater than C. The problem is to obtain the maximum amount of land which can be irrigated with these restrictions.

All parameters of the problem (length of the garden, position of the sprinklers, and flows) are integer values. Besides, all the sprinklers are situated inside the garden and at different positions.

Input

The input begins with a line where the number of test cases (T) is indicated. The data for each test case appear in successive lines.

In each test case, the first line contains the size of the garden (L, with maximum value 25). The second line contains the number of sprinklers (S, with maximum value 20). The third line contains S integers separated by spaces, and each integer represents the position of a sprinkler. The fourth line contains the maximum total flow (C, with maximum value 14). And the fifth line contains S integers separated by spaces, where each integer represents the maximum possible flow of the corresponding sprinkler $m[i]$, with maximum value 8).

You can assume that, for each sprinkler, its maximum flow will never irrigate any position outside the garden.

The input must be read from standard input.

Output

The output consists of a line for each problem. For each problem you have to output the maximum land space which can be irrigated taking into consideration the water flow restrictions.

The output must be written to standard output.

Sample Input	**Sample Output**
2 10 3 3 5 9 3 2 3 1 20 6 2 6 10 11 13 17 7 1 4 3 2 4 3	8 19